



**Universidad
Nacional
de San Martín**

UNIVERSIDAD NACIONAL DE SAN MARTÍN

EXAMEN PARCIAL DE PYTHON

FUNCIONES, ESTRUCTURAS DE DATOS Y ANÁLISIS DE DATOS

Instrucciones

- Lee cuidadosamente cada punto antes de programar.
- El examen dura 2 horas.
- Se evaluarán: funciones, listas, diccionarios, conjuntos, ciclos, condicionales, uso de `numpy`, `matplotlib` y `pandas`.
- Se descontarán puntos por errores de sintaxis, falta de claridad o no cumplir con el formato pedido de input y output.

1. Bloque 1 – Ejercicios de Python (Básico)

1. Conteo de vocales (8 pts)

Consigna: Implementa una función `contar_vocales(cadena)` que devuelva un diccionario con la cantidad de veces que aparece cada vocal.

Input: "programacion" **Output:** {"a":2, "o":2, "i":1}

2. Intersección de listas (8 pts)

Consigna: Implementa una función `interseccion(lista1, lista2)` que devuelva los elementos comunes entre ambas listas sin repetidos.

Input: [1,2,3,4], [3,4,5,6] **Output:** [3,4]

3. Palabra más frecuente (8 pts)

Consigna: Implementa `palabra_frecuente(lista)` que reciba una lista de palabras y retorne la palabra más repetida.

Input: ["sol","luna","sol","hipo"] **Output:** "sol"

4. Filtrar números primos (10 pts)

Consigna: Implementa `filtrar_primos(nums)` que devuelva una lista con los números primos de la lista original. Nota: un numero primo es aquel entero que solo puede dividirse por 1 y por si mismo.

Input: [2,3,4,5,6,7,8] **Output:** [2,3,5,7]

5. Diccionario de cuadrados (8 pts)

Consigna: Implementa `cuadrados(n)` que retorne un diccionario cuyas claves sean los enteros del 1 al n y los valores sus cuadrados.

Input: 5 **Output:** {1:1, 2:4, 3:9, 4:16, 5:25}

2. Bloque 2 – Numpy, Matplotlib y Pandas

6. Serie trigonométrica (8 pts)

Consigna: Genera 100 valores entre 0 y 2π . Calcula $\sin(x)$ y $\cos(x)$ y gráficlos en una misma figura con leyenda.

7. Gráfico de función cuadrática (8 pts)

Consigna: Genera 100 valores entre -10 y 10 . Calcula $y = x^2 - 4x + 3$ y grafica la función con título, etiquetas en los ejes. **Output esperado:** Gráfico de parábola.

8. Dispersión de datos simulados (8 pts)

Consigna: Genera 100 puntos aleatorios x entre 0 y 10. Calcula $y = 2x + \epsilon$, donde ϵ es ruido aleatorio normal con media 0 y desviación estándar 2. Grafica un diagrama de dispersión (scatter plot) de los puntos generados y añade la recta $y = 2x$ en color rojo para comparar. **Output esperado:** Gráfico de dispersión con nube de puntos y línea de referencia. —

9. Parcial de Pandas – League of Legends (34 pts totales)

Se cuenta con un archivo de Excel llamado `champions.xlsx` que contiene estadísticas base de los campeones de League of Legends.

El archivo tiene las siguientes columnas: `Champion Name`, `Role`, `Base Health`, `Base Mana`, `Base Armor`, `Base Attack Damage`, `Gold Efficiency`

1. Cargar el archivo en un DataFrame llamado `df`. (2 pts)
2. Eliminar filas duplicadas. (2 pts)
3. Eliminar filas con valores nulos (NaN). (2 pts)
4. Crear la columna `Recurso`:
 - “No usa maná” si `Base Mana == 0`
 - “Usa maná” en caso contrario. (4 pts)
5. Crear la columna `Tankiness Index = Base Health * Base Armor`. (4 pts)
6. Renombrar todas las columnas al español: (4 pts)
 - `Champion Name` → `Campeón`
 - `Role` → `Rol`
 - `Base Health` → `vida_base`
 - `Base Mana` → `mana_base`
 - `Base Armor` → `armadura_base`
 - `Base Attack Damage` → `danio_de_ataque_base`
 - `Gold Efficiency` → `eficiencia_de_oro`
7. Guardar el DataFrame como Excel con nombre `campeones_lol.xlsx`. (2 pts)
8. Agrupar por `Rol` y calcular la media de `Tankiness Index`. Mostrar en un nuevo DataFrame `df_roles`. (6 pts)

9. Graficar un gráfico de barras con **Eficiencia de Oro** en función de cada Campeón. (4 pts)
10. Mostrar solo las columnas **Campeón**, **Rol**, **Daño de Ataque Base** para campeones cuyo Rol sea “ADC” o “Jungle”. (4 pts)